Ćwiczenie 1 — Pierwszy delegat

Zadeklaruj delegat o nazwie Transformacja, który przyjmuje jeden parametr int i zwraca int.

Następnie:

- Napisz metodę Podwoj, która zwraca wartość × 2.
- Napisz metodę Neguj, która zwraca wartość × (-1).
- W metodzie Main przypisz kolejno obie metody do delegata i wywołaj je z liczbą 7.

Oczekiwane wyjście:

```
Podwoj(7) = 14
Neguj(7) = -7
```

Ćwiczenie 2 — Delegat jako parametr metody

Napisz metodę:

```
static void Zastosuj(int[] tablica, Transformacja t)
```

Metoda powinna przejść przez każdy element tablicy, zastosować delegat t i wypisać wynik na konsoli.

W Main utwórz tablicę { 1, 2, 3, 4, 5 } i wywołaj Zastosuj dwukrotnie — raz z metodą Podwoj, raz z Neguj.

Oczekiwane wyjście (fragment):

```
Podwoj: 2 4 6 8 10
Neguj:  -1 -2 -3 -4 -5
```

Ćwiczenie 3 — Delegat multicast: logger

Zadeklaruj delegat:

```
delegate void Logger(string wiadomosc)
```

Napisz trzy metody: LogKonsola, LogPlik (niech symuluje zapis — wypisuje "[PLIK] ...") oraz LogDebug ("[DEBUG] ...").

W Main połącz wszystkie trzy w jeden multicast delegat i wywołaj go z wiadomością "Aplikacja uruchomiona". Następnie usuń LogDebug i wywołaj ponownie z "Krok 2".

Ćwiczenie 4 — Wybór operacji przez użytkownika

Zadeklaruj delegat:

```
delegate double Obliczenie(double a, double b)
```

Zaimplementuj cztery metody: Dodaj, Odejmij, Mnoz, Dziel.

Program powinien:

1. Wczytać dwie liczby double od użytkownika.
2. Wyświetlić menu: [+] [-] [*] [/].
3. Przypisać odpowiednią metodę do delegata na podstawie wyboru (użyj switch).
4. Obsłużyć dzielenie przez zero — wypisz komunikat błędu bez rzucania wyjątku.
5. Wypisać wynik.

Ćwiczenie 5 — Delegat multicast z wartością zwracaną

Zadeklaruj delegat:

```
delegate int Przetwarzanie(int n)
```

Napisz cztery metody: Podwoj, Potrojn, DodajDziesiec, OdjijTrzy.

Połącz wszystkie cztery w jeden multicast delegat.

Wywołaj delegat z wartością 5 i zapisz wynik. Przypomnij sobie: kiedy delegat multicast zwraca wartość, zwracany jest tylko wynik ostatniej metody w łańcuchu — zweryfikuj to eksperymentalnie i wytłumacz w komentarzu, dlaczego tak jest.

Następnie wypisz wyniki każdej metody osobno, iterując po GetInvocationList().